

Sincronizar schema: o que a tarefa 4.1 não consegue medir

A tarefa pede pra conferir se as migrations 004–007 foram aplicadas em gc-dev e gc-prod. Existem migrations até a 010, não havia registro de nada do que já rodou, e os dois critérios de aceite passam hoje — com os types desatualizados. A verificação em dev já foi feita: está tudo aplicado, e sobra um item.

REPOSITÓRIO	BRANCH	COMMIT LIDO
cremalet/gc-sistema	feature-dev-breno-automacao	e2dcc57
VERIFICADO EM		
2026-09-02		

ESTADO

gc-dev: 004–010 aplicadas, 21 de 21 checagens OK, types regenerados. gc-prod está pausado — só dev está em uso, então tudo relativo a prod aqui é pendência registrada, não bloqueio. O veredito de "está aplicado" vem do script de verificação rodado no SQL Editor de dev; o de "types em dia" vem do db:types rodado contra o projeto.

Verificação em gc-dev

MIGRATION	CHECAGENS	RESULTADO	EVIDÊNCIA NO CATÁLOGO
004	8 / 8	APLICADA	clientes , cliente_id NOT NULL, FK, zero colunas legacy em obras e orcamentos , rename observacao , 4 views
005	2 / 2	APLICADA	RLS + 4 polices em clientes
006	1 / 1	APLICADA	calcular_valoresobra com o CASO 2 (fonte=sem_proposta)
007	1 / 1	APLICADA	progresso_itens_pct exposto na view
008	4 / 4	APLICADA	tabela, RLS, 4 polices, 3 índices

MIGRATION	CHECAGENS	RESULTADO	EVIDÊNCIA NO CATÁLOGO
009	3 / 3	APLICADA	tabela, RLS, 4 policies
010	2 / 2	APLICADA	bucket privado 20 MB <code>application/pdf</code> + 4 policies de <code>storage.objects</code>
prod	—	PAUSADO	Projeto fora do ar; verificação e aplicação em espera

Contexto do catálogo de dev: 16 tabelas e 53 policies em `public` . As 16 são exatamente as 14 que o `types.ts` já conhece mais `documentos_processamento` e `contatos_whatsapp` — o schema completo.

Os types já foram regenerados. O `npm run db:types` rodou contra `gzbmhgnpoehornidm` e trouxe os três objetos que faltavam, o que também prova que esse project-id é o `gc-dev`. O `types.ts` saiu de 2267 para 2449 linhas (+192/-10): entraram `contatos_whatsapp` com 8 colunas, `documentos_processamento` com 13, e `progresso_itens_pct` em `obras_com_valores` ; as 10 remoções são reformatação do gerador nos helpers genéricos. `tsc --noEmit` e `lint` seguem limpos. Dos cinco problemas a seguir, escopo, aplicação e drift de types estão resolvidos *em dev* — por evidência, não por suposição. Restam os dois estruturais.

O diagnóstico original, do mais grave pro menos

#	GRAVIDADE	PROBLEMA	IMPACTO	SITUAÇÃO
1	CRÍTICO	004 é destrutiva, não idempotente e sensível à ordem	Aplicada em prod sem backup/janela, dropa 13 colunas de <code>obras</code> com dados. Fora de ordem, reverte a 007 e derruba <code>/obras</code> .	DIFERIDO
2	CRÍTICO	Escopo desatualizado: são 004-010, não 004-007	008, 009 e 010 entraram depois da tarefa ser criada. A 010 é pré-requisito da ingestão via WhatsApp em prod.	RESOLVIDO EM DEV
3	ALTO	Não existe controle de migrations	Sem <code>supabase/migrations</code> , sem CLI, sem tabela de controle: "rodar as pendentes" não corresponde a comando nenhum.	ABERTO

#	GRAVIDADE	PROBLEMA	IMPACTO	SITUAÇÃO
4	ALTO	Critérios de aceite são falso-positivos	tsc e lint já passam com os types velhos. Um as unknown as apaga o drift.	ABERTO
5	MÉDIO	types.ts está três objetos atrás do schema	E db:types tem o --project-id fixo: não dá pra gerar os types do outro ambiente.	RESOLVIDO EM DEV

1

A 004 é destrutiva, não idempotente e sensível à ordem

A 004_revisao_schema.sql cria a tabela clientes , migra os dados de cliente que moravam em obras , e então dropa as colunas antigas — sem if not exists na criação e sem if exists nos drops. Ela está envolvida em begin/commit , então uma segunda execução aborta inteira em vez de deixar meio caminho. O risco não é re-rodar: é rodar a primeira vez em produção.

MIGRATION	IDEMPOTENTE	EM TRANSAÇÃO	POR QUÊ
004	não	sim	create table e drop column sem guard; migra dados de obras pra clientes
005	sim	não	drop policy if exists antes de cada create policy
006	sim	n/a	create or replace function , statement único
007	sim	n/a	create or replace view , statement único
008	não	não	create table documentos_processamento sem guard, seguida de índices, trigger e 4 policies
009	não	não	create table contatos_whatsapp sem guard, mesma estrutura
010	sim	não	on conflict do nothing no bucket + drop policy if exists

O que a 004 remove, em definitivo e sem backup: de obras , as colunas cliente , contato , cpf_cnpj , telefone , email , valor_total , desconto , valor_final , os quatro pct_* , forma_pagamento e doc_url —

treze no total; de `orcamentos` , outras seis. O cabeçalho do próprio arquivo pede janela de manutenção. O checklist da tarefa diz apenas "aplicar as mesmas migrations em gc-prod": não menciona backup nem janela.

A ordem não é sugestão

004

revisão de schema
recria as 4 views

→

005

RLS + policies
de clientes

→

006

conserta
calcular_valores_obra

→

007

progresso_itens_pct
na view

008

documentos_
processamento

→

009

contatos_
whatsapp

→

010

bucket + policies
de storage

004 DEPOIS DE 007 REVERTE A 007

A 004 recria `obras_com_valores` no final (linha 536) *sem* `progresso_itens_pct` . A listagem seleciona essa coluna por nome em `src/app/(app)/obras/page.tsx:51` — a página quebra em runtime, não em build.

004 SEM 006 DERRUBA A LISTAGEM INTEIRA

A 004 dropa os campos legacy que a `calcular_valores_obra` assumia, e a view chama a função via `LATERAL` . Aplicar a 004 em prod e parar ali deixa `/obras` fora do ar.

2

O escopo real é 004–010

Três migrations entraram no último commit (`e2dcc57` – migration e `documetacao`), depois da tarefa ter sido criada:

supabase/004_revisao_schema.sql

supabase/005_rls_clientes.sql

supabase/006_fix_calcular_valores_obra.sql

supabase/007_progresso_itens_obra.sql

supabase/008_documentos_processamento.sql ← e2dcc57

supabase/009_contatos_whatsapp.sql ← e2dcc57

supabase/010_storage_bucket_documentos_processamento.sql ← e2dcc57

A 010 é a mais fácil de esquecer e a mais barulhenta quando falta: ela cria o bucket `documentos-processamento` e as policies de `storage.objects` que exigem o `empresa_id` como primeiro segmento do path. O workflow n8n de ingestão via WhatsApp já grava em `{empresa_id}/whatsapp/{messageId}-{fileName}` contando com isso. A verificação confirmou a 010 aplicada em `dev`, então a ingestão está de pé no ambiente em uso; em prod é pendência registrada — e do tipo que

nenhum item do checklist original detectaria, porque storage não aparece nos types nem no compilador.

3 Ninguém sabe o que já rodou — e não é descuido, é ausência de mecanismo

Não existe `supabase/migrations/`, não existe `supabase/config.toml`, não existe projeto linkado pela CLI. Os `.sql` são scripts numerados aplicados à mão no SQL Editor. Daí decorrem três coisas:

- não há tabela de controle (`schema_migrations`) pra consultar;
- "rodar as migrations pendentes em gc-dev" não corresponde a comando nenhum;
- a única forma de conferir é **inspecionar o catálogo do Postgres**.

Foi pra isso que escrevi `supabase/verificar_schema_aplicado.sql`: somente leitura, seguro pra rodar em produção. Cola no SQL Editor de cada projeto e sai uma linha **OK** ou **FALTA** por evidência — tabela `clientes`, colunas legacy dropadas, FK `obras_cliente_fk`, as 4 views recriadas, RLS e contagem de policies, o CASO 2 da `calcular_valoresobra`, a coluna `progresso_itens_pct`, as duas tabelas novas, o bucket e suas policies — mais contagens totais de tabelas e policies pra diff direto entre dev e prod.

4 Os critérios de aceite não podem falhar

Rodados nesta branch, agora, com os types desatualizados:

```
$ npx tsc --noEmit
(sem erros)

$ npm run lint
✓ No ESLint warnings or errors
```

Os itens 4 e 5 do checklist já estão verdes *apesar* do drift, por dois motivos concretos: `src/lib/types.ts:232` declara `progresso_itens_pct` à mão, e `src/app/(app)/obras/page.tsx:96` faz `(data ?? [])` as unknown as `ObraListItem[]`. O `as unknown as` apaga qualquer divergência entre a string do `select()` e o `Database` gerado. Enquanto ele existir, o `tsc` nunca vai apontar coluna faltando — nem agora, nem na próxima migration.

5 O drift de types, objeto por objeto

`src/lib/supabase/types.ts` tem 2.267 linhas e não contém nenhum destes três:

MIGRATION 007 obras_com_valores .progresso_itens_pct ausente da view tipada	MIGRATION 008 documentos_processamento tabela ausente	MIGRATION 009 contatos_whatsapp tabela ausente
---	---	--

E o script de geração aponta pra um projeto fixo, sem chave de ambiente:

```
// package.json — como estava
"db:types": "npx --yes supabase gen types typescript
               --project-id gzbmhgnpoehormnidmgg > src/lib/supabase/types.ts"

// como está agora: gera num .tmp e só promove se der certo
"db:types": "npx --yes supabase gen types typescript
               --project-id ${SUPABASE_PROJECT_ID:-gzbmhgnpoehormnidmgg}
               > src/lib/supabase/types.ts.tmp
               && mv src/lib/supabase/types.ts.tmp src/lib/supabase/types.ts
               || (rm -f src/lib/supabase/types.ts.tmp; exit 1)"
```

O repositório não documenta se `gzbmhgnpoehormnidmgg` é `gc-dev` ou `gc-prod`, e não há como gerar os types do outro ambiente. Caminho: dois scripts (`db:types:dev` / `db:types:prod`) ou `--project-id $SUPABASE_PROJECT_ID`.

Cuidado, e já mordeu: o script redirecionava com `>` direto pro arquivo final, e o shell trunca o destino *antes* de rodar o comando. Na primeira tentativa a CLI abortou por falta de token (`supabase login`) e o `src/lib/supabase/types.ts` ficou com 0 bytes — restaurado do commit `e2dcc57` (blob `ab59e07`, 2267 linhas). O script agora escreve num `.tmp` e só promove com `mv` se a geração terminar bem, então uma falha deixa o arquivo bom intacto.

Com dev verificado, rodar `npm run db:types` agora responde essa pergunta de graça: se os três objetos ausentes aparecerem no diff, o id é `gc-dev` e o atraso era só falta de regenerar. Se não aparecerem — ou o comando falhar por projeto indisponível — o id aponta pra `gc-prod`, que está pausado, e o script precisa mudar antes de qualquer outra coisa.

Checklist — estado em 2026-09-02

Feito

verificado em dev

- 01 ☒ Rodar `supabase/verificar_schema_aplicado.sql` em `gc-dev` — 21/21 OK
- 02 ☒ Confirmar que 004–010 estão aplicadas em dev, por evidência de catálogo

Dev — feito, e o que restou

passos 3–7

- 03 ☒ Autenticar a CLI — feito com personal access token via `SUPABASE_ACCESS_TOKEN`; o fluxo de navegador não roda sem TTY
- 04 ☒ `npm run db:types` — os três objetos apareceram, e `gzbmhgnpoehornnidmgg` é o `gc-dev`
- 05 ☒ `npx tsc --noEmit` e `npm run lint` com os types novos — ambos limpos
- 06 ☒ `db:types` blindado contra truncamento e aceitando `$SUPABASE_PROJECT_ID`, com `db:types:dev` ao lado
- 07 ☒ 008 e 009 envolvidas em `begin; ... commit;` — não muda nada em dev, protege a aplicação futura em prod

Pendente de quem tem acesso

um comando

- 13 ☐ `supabase migration repair --status applied` nas dez versões, pra o histórico da CLI refletir o que já está em dev — sem isso um `db push` tentaria reaplicar a `revisao_schema` (ex-004)

Em espera — gc-prod pausado

passos 8–10

- 08 ☐ Rodar o script de verificação em `gc-prod` e diffar contra a saída de dev
- 09 ☐ Backup + janela de manutenção antes de qualquer coisa em prod — a 004 dropa 13 colunas de `obras` e é irreversível
- 10 ☐ Aplicar o que faltar em prod na ordem 004→010 e revalidar com o script

Estrutural — feito

passos 11-12

- 11 ☒ as unknown as removido das três listagens (obras , fd , orcamentos) — um as só mantém o narrowing e devolve ao tsc a checagem contra o Database
- 12 ☒ supabase/migrations + config.toml adotados; as dez migrations viraram arquivos com timestamp e o procedimento está em supabase/README.md

Migrations sob a CLI

Os dez scripts numerados viraram um histórico que a CLI entende. Os timestamps são as datas reais dos commits que introduziram cada arquivo, então a ordem cronológica é idêntica à da numeração antiga — e a numeração ficou anotada no supabase/README.md , porque a documentação e as tarefas do ClickUp falam nesses números.

ANTES	AGORA, EM SUPABASE/MIGRATIONS/
001	20260424121550_initial.sql
002	20260424121551_rls_policies.sql
003	20260424121552_storage_buckets.sql
004	20260511151924_revisao_schema.sql DESTRUTIVA
005	20260512135125_rls_clientes.sql
006	20260512141810_fix_calcular_valores_obra.sql
007	20260512151918_progresso_itens_obra.sql
008	20260901120909_documentos_processamento.sql
009	20260901120910_contatos_whatsapp.sql
010	20260901120911_storage_bucket_documentos_processamento.sql

Os scripts que não são migrations ficaram fora de migrations/ e nunca entram num db push : 000_reset_dev.sql , setup_inicial_dev.sql , seed_orcamentos.sql e o verificar_schema_aplicado.sql .

Falta um comando, e ele importa. As dez já estão aplicadas em dev, mas o histórico remoto da CLI está vazio — ela não tem como saber. Antes de qualquer `db push`, é preciso rodar `migration repair --status applied` nas dez versões; senão a CLI tenta reaplicar tudo a partir da `revisao_schema`, que é destrutiva. Ela aborta sozinha (está em `begin/commit`) e nada é perdido, mas o susto é evitável. Os comandos exatos estão em `supabase/README.md`.

O que está entregue

`supabase/migrations/` · `supabase/config.toml` · `supabase/README.md`

As dez migrations sob o histórico da CLI, a config mínima que a própria CLI validou, e o runbook com o `link`, o `repair` e as duas regras que o histórico deste repo justifica (transação sempre; cuidado com view recriada em migration nova).

`supabase/verificar_schema_aplicado.sql`

Uma única query, somente leitura, sobre os catálogos do Postgres. Cobre 004 a 010 com ~20 evidências e devolve `migration · status · checagem · esperado · encontrado` ordenado por migration. Seguro em produção: não há DDL nem DML. Validado como Postgres antes de entregar.

`docs/4.1-sincronizar-schema-analise.md`

Esta análise em markdown, versionada junto do código, com o checklist por estado.

`docs/4.1-sincronizar-schema.html` · `docs/4.1-sincronizar-schema.pdf`

A página e o PDF (A4) gerados do mesmo HTML. Tudo está commitado na branch `worktree-verificar-schema-4-1`, que parte de `e2dcc57` e por isso entra em `feature-dev-breno-automacao` como fast-forward. O push pro GitHub falhou com 403 — `brenocml` não tem permissão de escrita em `cremalet/gc-sistema`, então os commits existem só no clone local.

Diagnóstico por leitura estática do repositório

Dev verificado no catálogo em 2026-09-02 · types regenerados · migrations sob a CLI · gc-prod pausado

`e2dcc57` · `feature-dev-breno-automacao`